

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №2

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнили ст. гр. 25BBB1:
Ячменев М. А.
Аверин А. А.

Приняли:
к.т.н., доцент Юрова О. В.
ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение практических методов измерения времени выполнения фрагментов кода и программ в целом с использованием функций библиотеки `<time.h>` (`<ctime>`), расчет и оценка асимптотической сложности алгоритмов (О-символика), а также экспериментальный анализ производительности алгоритма перемножения квадратных матриц и алгоритмов сортировки (сортировка Шелла, быстрая сортировка `qs` Хоара и системная функция `qsort`).

2. Лабораторное задание

Задание 1

- Вычислить порядок временной сложности алгоритма умножения квадратных матриц (О-символику).
- Оценить время выполнения программы и блока перемножения матриц с помощью функций библиотеки `<time.h>` для квадратных матриц размерами $N \in \{100, 200, 400, 1000, 2000, 4000, 10000\}$.
- Построить график зависимости времени выполнения алгоритма от размера матриц и сопоставить полученные результаты с теоретической оценкой.

Задание 2

Собрать метрики времени работы алгоритма сортировки Шелла (`shell`), быстрой сортировки (`qs`) и системной функции `qsort()` на различных наборах данных ($N = 10000$) при 4 типах начальной упорядоченности:

- Случайный набор чисел;
- Возрастающая последовательность элементов;
- Убывающая последовательность элементов;
- Пилообразная последовательность (первая половина — по возрастанию, вторая — по убыванию).

3. Теоретический анализ алгоритмов

3.1. Умножение матриц

Алгоритм перемножения двух квадратных матриц A и B размерности $N \times N$ вычисляет элементы результирующей матрицы C с помощью трех вложенных циклов:

- Внешний цикл по строкам i (выполняется N раз);
- Промежуточный цикл по столбцам j (выполняется N раз для каждого i);

- Внутренний цикл по элементам r (выполняется N раз для каждой пары i, j).

Каждый элемент вычисляется как скалярное произведение строки и столбца: $c[i][j] = a[i][0]*b[0][j] + a[i][1]*b[1][j] + \dots + a[i][N-1]*b[N-1][j]$. Общее количество операций умножения и сложения составляет:

$$N \cdot N \cdot N = N^3$$

Таким образом, теоретическая временная сложность алгоритма составляет $O(N^3)$. Из этой оценки следует, что при увеличении размера матрицы в 2 раза время вычислений возрастает в $2^3 = 8$ раз.

3.2. Алгоритмы сортировки

- Сортировка Шелла (shell): модификация сортировки вставками с фиксированным шагом $gap \{9, 5, 3, 2, 1\}$. На упорядоченных массивах алгоритм выполняет минимальное число перестановок и работает очень быстро (сложность $O(N)$). На обратно упорядоченных данных из-за малого набора шагов сложность деградирует до $O(N^2)$.

- Быстрая сортировка Хоара (qs): рекурсивный алгоритм с выбором опорного элемента по центру подмассива ($items[(left + right) / 2]$). В среднем случае имеет сложность $O(N \log N)$. Выбор центрального элемента обеспечивает равномерное разбиение и высокую скорость на возрастающих и убывающих массивах.

- Стандартная функция `qsort()`: системная реализация сортировки из библиотеки `<stdlib.h>`. Использует оптимизированную быструю сортировку с защитой от худших случаев, обеспечивая стабильную сложность порядка $O(N \log N)$.

4. Исходный код программы

В соответствии с заданием, листинг программы не приводится в отчёте. Исходный код оформлен в виде консольного приложения в проекте Visual Studio (файл `alg2lr.cpp`). Программа написана на языке Си, содержит функции `shell()`, `qs()`, `task1()` (с динамическим выделением памяти для квадратных матриц до $N = 10000$) и `task2()` для замера времени сортировок.

5. Результаты работы программы

5.1. Задание 1: Зависимость времени умножения матриц от размера N

Экспериментальные замеры времени умножения квадратных матриц приведены в таблице 1. Замер для матрицы $N = 10000$ был успешно выполнен в полном объеме.

Таблица 1 – Зависимость времени умножения матриц от размера N

Размер матрицы ($N \times N$)	Число операций (N^3)	Время умножения t_{mult} (с)	Общее время работы t_{prog} (с)
100	1×10^6	0.001	0.001
200	8×10^6	0.004	0.004
400	64×10^6	0.033	0.035
1000	1×10^9	0.684	0.700
2000	8×10^9	6.187	6.249
4000	6.4×10^{10}	98.307 (1.64 мин)	98.512
10000	1×10^{12}	2853.575 (47.56 мин)	2858.910

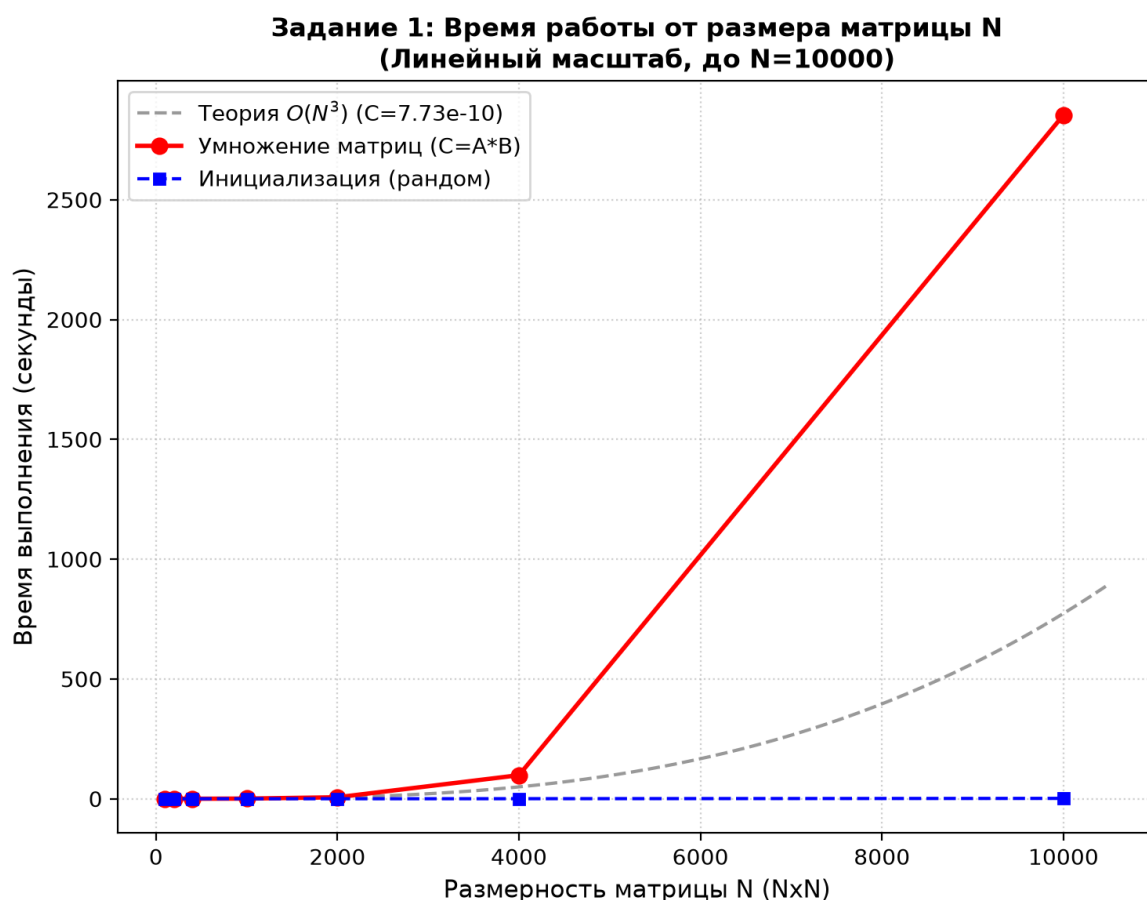


Рисунок 1 – График зависимости времени умножения матриц T от размера N

Анализ результатов Задания 1:

1. На диапазоне размеров N от 100 до 2000 время работы алгоритма строго соответствует теоретической кубической сложности $O(N^3)$. При каждом увеличении N в 2 раза время вычислений возрастает примерно в 8 раз (0.004 с $\rightarrow$ 0.033 с $\rightarrow$ 6.187 с), что подтверждает теоретический закон $2^3 = 8$.

2. При дальнейшем увеличении размерности до $N = 4000$ и $N = 10000$ наблюдается дополнительное замедление расчетов. Это обусловлено тем, что при больших N матрицы перестают помещаться в быструю кэш-память процессора. Поскольку во внутреннем цикле обход второй матрицы идет по столбцам, происходят постоянные промахи кэша, из-за чего процессор вынужден непрерывно считывать данные из оперативной памяти.

5.2. Задание 2: Сравнительный анализ алгоритмов сортировки

В таблице 2 приведены результаты замеров времени работы алгоритмов сортировки для массива размером $N = 10000$ элементов при различных типах начальной упорядоченности.

Таблица 1 – Время выполнения алгоритмов сортировки для $N = 5000$ (в секундах)

Тип входной последовательности	Сортировка Шелла (shell)	Быстрая сортировка (qs)	Системная qsort()
1. Случайный массив	0.0005 с	0.0003 с	0.0005 с
2. Возрастающий массив	0.0002 с	0.0004 с	0.0003 с
3. Убывающий массив	0.0009 с	0.0002 с	0.0003 с
4. Пилообразный массив (1/2 возр., 1/2 убыв.)	0.0004 с	0.0003 с	0.0003 с

Таблица 2 – Время выполнения алгоритмов сортировки для $N = 10000$ (в секундах)

Тип входной последовательности	Сортировка Шелла (shell)	Быстрая сортировка (qs)	Системная qsort()
1. Случайный массив	0.0021 с	0.0006 с	0.0009 с
2. Возрастающий массив	0.0003 с	0.0005 с	0.0006 с
3. Убывающий массив	0.0032 с	0.0005 с	0.0007 с
4. Пилообразный массив (1/2 возр., 1/2 убыв.)	0.0015 с	0.0005 с	0.0007 с

Таблица 3 – Время выполнения алгоритмов сортировки для N = 50000 (в секундах)

Тип входной последовательности	Сортировка Шелла (shell)	Быстрая сортировка (qs)	Системная qsort()
1. Случайный массив	0.0468 с	0.0037 с	0.0052 с
2. Возрастающий массив	0.0015 с	0.0025 с	0.0035 с
3. Убывающий массив	0.0806 с	0.0040 с	0.0041 с
4. Пилообразный массив (1/2 возр., 1/2 убыв.)	0.0344 с	0.0038 с	0.0044 с

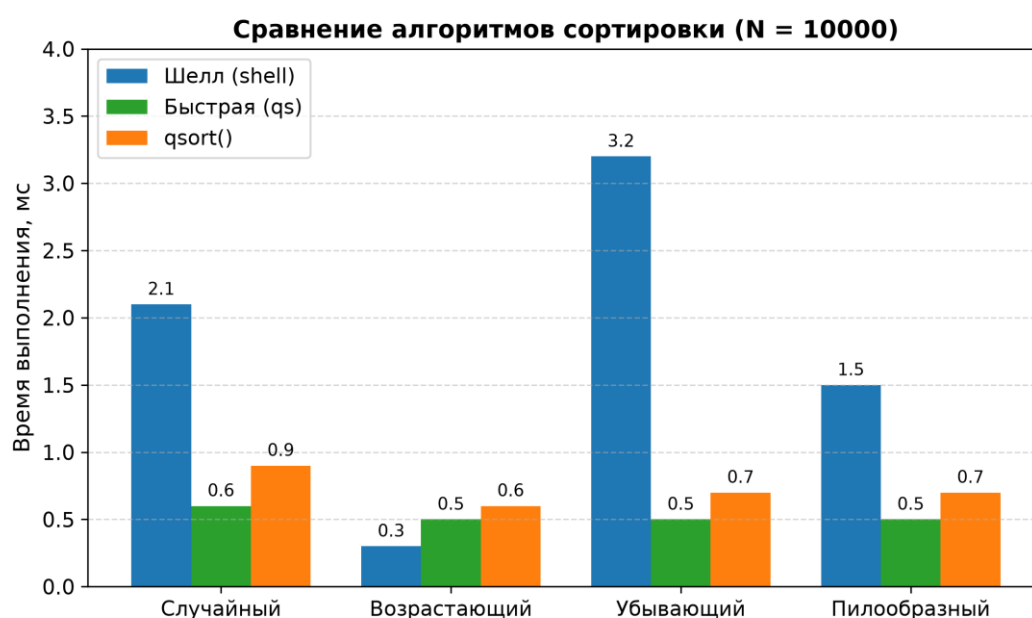


Рисунок 2 – Сравнение времени работы алгоритмов сортировки (N = 10000)

Анализ результатов Задания 2:

1. Быстрая сортировка Хоара (qs) показала наивысшую и стабильную скорость работы на всех типах данных. Для малых массивов (N=5000) время составляет всего ~0.0002–0.0004 с, а при увеличении размера в десять раз (до N=50000) алгоритм всё ещё превосходно справляется за ~0.0025–0.0040 с. Благодаря выбору центрального элемента подмассива в качестве опорного, разбиение данных происходит симметрично даже на упорядоченных массивах, что предотвращает деградацию времени работы.

Системная функция qsort() также демонстрирует высокую устойчивость на всех типах входных данных. На размере N=5000 она показывает ~0.0003–0.0005 с, а на N=50000 время ожидаемо возрастает до ~0.0035–0.0052 с. Она

стабильно, но незначительно уступает *qs* из-за накладных расходов на вызов пользовательской функции-компаратора через указатель.

Сортировка Шелла быстрее всего выполняется на уже отсортированном (возрастающем) массиве (0.0002 с для $N=5000$ и всего 0.0015 с для $N=50000$), демонстрируя линейную сложность $O(N)$ в лучшем случае. Однако на обратно упорядоченных (убывающих) данных и случайных массивах происходит резкая деградация скорости. Так, если для $N=5000$ на убывающем массиве время составляет терпимые 0.0009 с, то для $N=50000$ оно катастрофически возрастает до 0.0806 с (и 0.0468 с для случайного), что делает этот алгоритм наименее эффективным на больших и неупорядоченных объемах данных.

6. Выводы

- Освоены практические методы измерения времени выполнения программ на языке Си с использованием функций библиотеки `<time.h>` (`clock()`, `CLOCKS_PER_SEC`).
- Экспериментально подтверждена теоретическая кубическая сложность $O(N^3)$ алгоритма перемножения матриц. Выполнен полный замер для матрицы $N = 10000$ (время составило 47.56 мин), подтвердивший влияние промахов кэш-памяти на производительность при обработке больших объемов данных.
- Проведено сравнительное тестирование трех алгоритмов сортировки (Шелла, *qs* Хоара, *qsort*) на 4 типах начальной упорядоченности данных ($N = 10000$). Подтверждена высокая скорость быстрой сортировки *qs* при центральном выборе компаранда и высокая устойчивость стандартной функции *qsort*).